Experiment 8

N-Queens Problem Using Backtracking

Aim

To write a JAVA program to solve the N-Queens problem using the Backtracking technique, display the total number of distinct solutions for a given value of N, and display the position of each queen on the chessboard for every valid solution while ensuring that no two queens attack each other.

Theory

The N-Queens problem is a classic combinatorial problem in computer science in which the objective is to place N queens on an $N \times N$chessboard such that no two queens attack each other. Since a queen can move horizontally, vertically, and diagonally, no two queens can occupy the same row, column, or diagonal.

Backtracking is an algorithmic technique that builds a solution step by step and abandons a partial solution whenever it violates the given constraints. In the N-Queens problem, the algorithm places one queen in each row, starting from the first row. For every row, it checks each column to determine whether the position is safe. If the position is safe, the queen is placed, and the algorithm proceeds to the next row. If no safe position is available, the algorithm removes the previously placed queen and tries another position.

The program uses arrays to track occupied columns and diagonals, making safety checks efficient. Whenever all N queens are placed successfully, the arrangement is recorded and displayed as a chessboard, along with the positions of the queens. The process continues until all possible valid arrangements have been explored, and the total number of distinct solutions is displayed.

The N-Queens problem demonstrates the practical application of recursion, constraint checking, and systematic searching. Its worst-case time complexity is $O(N!)$, while the space complexity is $O(N^2)$due to the chessboard and $O(N)$auxiliary space for tracking arrays and recursion.

Algorithm

1. Read the value of N from the user.
2. Create an $N \times N$chessboard initialized with empty positions.
3. Maintain arrays to track occupied columns and diagonals.
4. Start placing queens from the first row.
5. For each row, try placing a queen in every column.
6. Check whether the selected position is safe by verifying that its column and both diagonals are unoccupied.
7. If the position is safe, place the queen and recursively process the next row.
8. If all N queens are placed, increment the solution count and display the chessboard.
9. After exploring a position, remove the queen and mark its column and diagonals as available to explore other possibilities.
10. Repeat until all possible arrangements have been examined.
11. Display the total number of distinct solutions for the given N.

Program / Code

```
import java.util.Scanner;

public class Main {
    static int n;
    static int[][] board;
    static boolean[] columns;
    static boolean[] diag1;
    static boolean[] diag2;
    static int count = 0;

    static void solve(int row) {
        if (row == n) {
            count++;
            System.out.println("\nSolution " + count + ":");
            displayBoard();
            displayPositions();
            return;
        }

        for (int col = 0; col < n; col++) {
            int d1 = row - col + n - 1;
            int d2 = row + col;

            if (!columns[col] && !diag1[d1] && !diag2[d2]) {
                board[row][col] = 1;
                columns[col] = true;
                diag1[d1] = true;
                diag2[d2] = true;

                solve(row + 1);

                board[row][col] = 0;
                columns[col] = false;
                diag1[d1] = false;
                diag2[d2] = false;
            }
        }
    }

    static void displayBoard() {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(board[i][j] == 1 ? "Q " : ". ");
            }
            System.out.println();
        }
    }

    static void displayPositions() {
        System.out.print("Queen positions (row, column): ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == 1) {
                    System.out.print("(" + (i + 1) + ","
                                     + (j + 1) + ") ");
                }
            }
        }
        System.out.println();
    }
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the value of N: ");
        n = sc.nextInt();

        if (n <= 0) {
            System.out.println("N must be a positive integer.");
            sc.close();
            return;
        }

        board = new int[n][n];
        columns = new boolean[n];
        diag1 = new boolean[2 * n - 1];
        diag2 = new boolean[2 * n - 1];

        solve(0);

        System.out.println("\nTotal distinct solutions: " + count);
        sc.close();
    }
}
```

Output

```
Enter the value of N: 4

Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .
Queen positions (row, column): (1,2) (2,4) (3,1) (4,3)

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .
Queen positions (row, column): (1,3) (2,1) (3,4) (4,2)

Total distinct solutions: 2
```

Conclusion

The N-Queens problem was successfully implemented in Java using the Backtracking technique. The program accepted the value of N from the user and displayed all valid arrangements of queens while ensuring that no two queens shared the same row, column, or diagonal. For $N = 4$, the program found two distinct solutions and displayed their chessboard arrangements and queen positions.

The experiment demonstrated how Backtracking explores possible solutions, rejects invalid placements, and revisits previous decisions to find alternative arrangements. The use of Boolean arrays improved the efficiency of checking columns and diagonals. Thus, the experiment helped develop an understanding of recursion, constraint satisfaction, and algorithm complexity, as well as the application of Backtracking to problems such as Sudoku Solver and graph coloring.

Learning Outcomes

- Understood the N-Queens problem and its constraints involving rows, columns, and diagonals.
- Learned how to implement the Backtracking technique using recursion in Java.
- Learned to place queens incrementally and undo placements when a constraint is violated.
- Understood how to track occupied columns and diagonals to check the safety of queen placements efficiently.
- Learned to display every valid chessboard arrangement and calculate the total number of distinct solutions.
- Analysed the time and space complexity of the Backtracking algorithm.
- Recognised the application of Backtracking to other constraint satisfaction problems, such as Sudoku Solver and other combinatorial search problems.